

自然语言处理

王刚

统计与数据科学学院

2026 年春季学期

目录

第八章 机器翻译

8.1 机器翻译概述

8.2 基于统计的机器翻译方法

8.3 基于神经网络的机器翻译方法

8.4 机器翻译评估

8.5 机器翻译语料库

第八章 机器翻译

引言

- ▶ 根据联合国统计，目前世界上正在使用的语言约有 6000 种，教育系统和公共领域中使用到的语言也有数百种之多。我们不可能掌握如此之多的语言，即便是熟练使用联合国规定的包括阿拉伯语、汉语、英语、法语、俄语和西班牙语在内的六种正式语言都非常困难。
- ▶ 因此，第一台通用计算机 ENIAC 才刚刚面世，Warren Weaver 就在 1947 年提出了利用计算机翻译人类语言的可能。
- ▶ 机器翻译 (Machine Translation) 是指利用计算机将一种语言 (源语言) 自动翻译为另外一种语言 (目标语言) 的过程。机器翻译是自然语言处理中研究历史最长，也是最重要的任务之一。

引言

Google 翻译

Google Translate interface showing the translation of the Chinese text "人要是行，干一行行一行，一行行行行行，行行行干哪行都行。要是不行，干一行不行一行，一行不行行行不行，行行行干哪行都不行。" into English: "If a person can do it, he can do it in a row, in a row, in a row, and he can do whatever he wants. If you can't do it, you can't do it in one line, you can't do it in one line, you can't do it in one line, you can't do it in one line." The interface includes tabs for Text, Images, Documents, and Websites, and a dropdown menu for selecting the source and target languages.

Baidu 翻译

Baidu Translate interface showing the same Chinese text translated into English: "If a person is capable, they can do whatever they want. If it doesn't work, one line won't work, one line won't work, and no line will work." The interface includes a search bar, a dropdown menu for selecting the source and target languages, and a button for "人工翻译" (Human Translation).

翻译：人要是行，干一行行一行，一行行行行行，行行行干哪行都行。要是不行，干一行不行一行，一行不行行行不行，行行行干哪行都不行。

ChatGPT

If a person is capable, they can do any job well, whether it's one job or many. But if they are not capable, they won't be able to do any job well, whether it's one job or many.

8.1 机器翻译概述

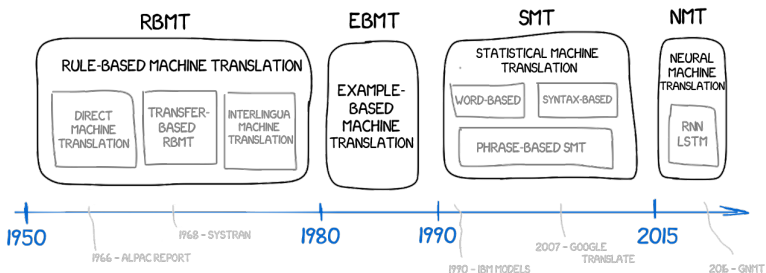
机器翻译概述

- ▶ 机器翻译 (Machine Translation, MT) 这一概念拥有很长的历史，相关领域的研究最早可以追溯到 17 世纪。1629 年 Descartes 等人就提出使用统一符号表达不同语言中的同一概念的语义。
- ▶ 现代机器翻译的研究始于上世纪五十年代，Bar-Hillel 等人在 1951 年就开始了对机器翻译的研究，并在 1952 年组织了第一届国际机器翻译会议 (International Conference on Machine Translation)。
- ▶ 机器翻译的任务定义相对简单，目标就是通过计算机将源语言 (Source Language) 翻译为目标语言 (Target Language)。

机器翻译发展历程

- ▶ 机器翻译的发展历程基本代表了自然语言处理领域的发展过程，迄今为止，机器翻译的研究与发展大体上经历了三次主要的浪潮：基于规则的机器翻译、基于统计的机器翻译以及基于神经网络的机器翻译。

A BRIEF HISTORY OF MACHINE TRANSLATION



基于规则的机器翻译

- ▶ 基于规则的机器翻译假设“每一种语义在不同的语言当中都存在与其相对应的符号”。
- ▶ 对于某种语言中的大多数单词而言，通常都能够在另一种语言当中找到表达相同含义的对应的单词。在这类方法当中，翻译过程通常被看作一个源语言的词替换过程。
- ▶ 基于规则的机器翻译方法的理论非常简洁清晰，但在实践中的性能却不尽如人意。这是由于选择与给定源语言相适配的句法规则在计算上非常低效。同时，为了应对多样的语言现象，语言学家们设计了规模庞大的句法规则。但是，这些规则很难被有效地组织，甚至会出现不同规则相互矛盾的情况。
- ▶ 基于规则的方法最严重的缺陷在于其缺乏翻译过程中对上下文信息的建模，这使得基于规则的翻译模型的鲁棒性不佳。

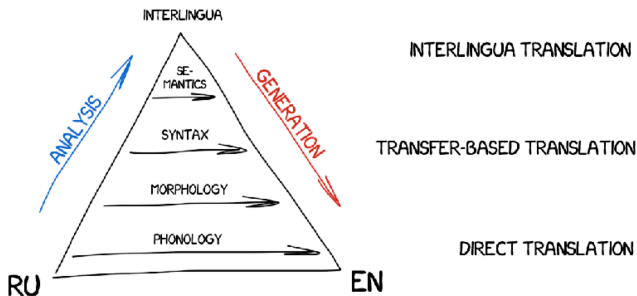
“The pen is in the box”

“The box is in the pen”

上述两个句子具有相同的句法结构，第一个句子很容易理解。但第二个句子非常令人困惑，由于在英文中“pen”本身是一个多义词，除了“笔”之外它还有“栅栏”的意思。但对于计算机来说很难直接将“pen”对应到“栅栏”的含义上，进而产生错误的翻译结果。

基于规则的机器翻译

- ▶ 基于规则的机器翻译有三种技术路线
 - 直接机器翻译 (Direct Machine Translation)
 - 转换机器翻译 (Transfer-based Machine Translation)
 - 语际机器翻译 (Interlingual Machine Translation)



基于规则的机器翻译

直接机器翻译 (Direct Machine Translation)

- ▶ 在机器翻译研究的初期，人们一般采用直接翻译的方法，从源语言句子的表层出发，将单词或者词组、短语甚至句子直接置换成目标语言译文，有时进行一些简单的词序调整。
- ▶ 在这种翻译方法中，对原文句子的分析仅仅满足于特定译文生成的需要。这类翻译系统一般针对某一个特定的语言对，将句子分析与生成、语言数据、文法和规则与程序等都融合在一起。
- ▶ 这是最直接的机器翻译类型：它把文本分成词，翻译出来，稍微修正词法，调和句法，其中语言学家为每个单词构造规则。但是通常情况下，结果是相当糟糕的。

基于规则的机器翻译

转换机器翻译 (Transfer-based Machine Translation)

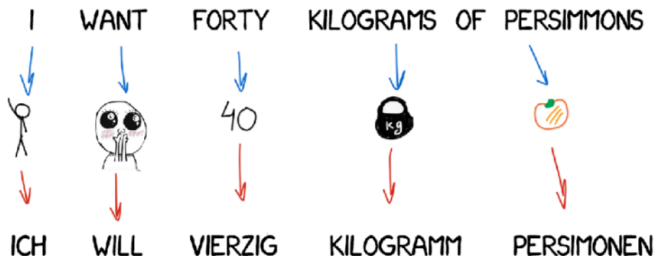
- ▶ 1957 年美国学者 V.Yingve 在《句法翻译框架》(Framework for Syntactic Translation) 一文中提出了对源语言和目标语言都进行适当描述、把翻译机制与语法分开、用规则描述语法的实现思想，这就是基于规则的转换翻译方法。其翻译过程分成三个阶段：
 - 1) 对输入文本进行分析，形成源语言抽象的内部表达；
 - 2) 将源语言内部表达转换成抽象的目标语言内部表达；
 - 3) 根据目标语言内部表达生成目标语言文本。
- ▶ 这种方法的主要环节可归纳为“独立分析—独立生成—相关转换”
 - a) 优点：可以较好地保持原文结构，产生的译文结构与原文结构关系密切，尤其对于语言现象已知或句法结构规范的源语言句子具有较强的处理能力和较好的翻译效果。
 - b) 不足：分析规则由人工编写，工作量大，规则的主观性强，规则的一致性难以保障，不利于系统扩充，尤其对非规范的语言现象缺乏相应的处理能力。

基于规则的机器翻译

语际机器翻译 (Interlingual Machine Translation)

- ▶ 基于中间语言 (interlingua-based) 的翻译方法首先将源语言句子分析成一种与具体语种无关的通用语言 (universal language) 或中间语言 (interlingua)，然后根据中间语言生成相应的目标语言。

整个翻译过程包括两个独立的阶段：从源语言到中间语言的转换阶段和从中间语言到目标语言的生成阶段。



基于规则的机器翻译

语际机器翻译 (Interlingual Machine Translation)

- ▶ 理论上，中间语言是逻辑化和形式化的语义表达语言，中间语言的设计可以不考虑具体的翻译语言对，因此，该方法尤其适用于多语言之间的互译。
- ▶ 但是，如何定义和设计中间语言的表达方式并不是一件容易的事情，中间语言在语义表达的准确性、完整性、鲁棒性和领域可移植性等诸多方面都面临很多困难，因此，基于中间语言的翻译方法在具体实现时受到了很大限制。

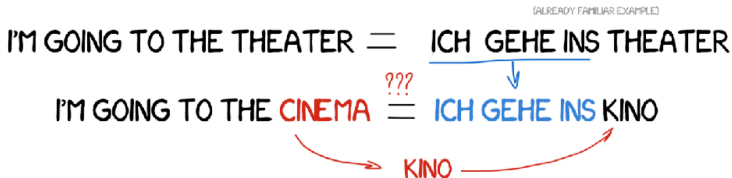
基于规则的机器翻译

- ▶ 早期机器翻译系统多为基于规则的翻译系统，需要由语言学家编写两种语言之间的转换规则，再将这些规则录入计算机。
- ▶ **RBMT 的优点**：是它的形态精确性 (不会混淆单词)、结果的再现性 (所有译者都得到同样的结果) 以及对于语言现象已知或者结构规范的源语言效果较好；
- ▶ **缺点**：人工编写工作量大、主观性强、难以保证一致性、不利于系统扩展、无法解释不规范的语言翻译。
- ▶ 该方法对语言学家的要求非常高，而且我们几乎无法总结一门语言会用到的所有规则，更何况两种甚至更多的语言；
- ▶ 因此，传统机器翻译方法面临的主要挑战是无法得到一个完备的规则集合。

基于规则的机器翻译

Example-based Machine Translation (EBMT)

- ▶ 由日本学者长尾真 (Makoto Nagao) 于 20 世纪 80 年代初期提出，但真正实现是在 80 年代末期。
- ▶ 该方法需要对已知语料进行词法、句法，甚至语义等分析，建立实例库用以存放翻译实例。系统在执行翻译过程时，首先对翻译句子进行适当的预处理，然后将其与实例库中的翻译实例进行相似性分析，最后，根据找到的相似实例的译文得到翻译句子的译文。



- ▶ 例子越多，译文就越好。只需要给机器输入现有的翻译，而不需要花费数年时间来编写规则。虽未有技术上的突破，但也有足够的进步。

基于统计的机器翻译

- ▶ 在过去的 20 年以来，统计机器翻译 (Statistical Machine Translation, SMT) 已经成为机器翻译领域的主流方法，并在工业界得到了广泛的实际应用。
- ▶ 与基于规则的方法不同，统计机器翻译完全从数据驱动的角度建模机器翻译任务。具体来说，通过对双语语料库的统计找到表达相同含义的单词或短语。给定一个源语言句子，统计机器翻译首先将其分割成若干个子句，接下来每个部分可以被目标语言的单词或短语替代。
- ▶ 统计机器翻译中最主流的方法是基于词的统计机器翻译 (Word-based MT) 以及基于短语的统计机器翻译 (Phrase-based SMT)，总体上来看包含预处理、句子对齐、词对齐、短语抽取、短语特征准备、语言模型训练等步骤。
- ▶ 基于短语的统计机器翻译方法的可以将源语言和目标语言中的短语配对。在这一过程中，翻译模型能够利用短语内部的上下文信息，因而优于简单的单词到单词的翻译方法。

基于神经网络的机器翻译

- ▶ 神经网络方法在机器翻译任务上的应用可以追溯到上世纪八九十年代。但受限于当时的计算资源和数据规模的限制，神经网络方法的性能差强人意，故而其发展停滞了很多年。
- ▶ 近年来，伴随着深度学习技术的广泛应用，越来越多的自然语言处理任务实现了极大的性能提升。基于深度神经网络的机器翻译模型也逐渐受到了许多关注。[神经网络方法在机器翻译任务上的第一次成功应用是基于递归神经网络的方法](#)，这在当时的机器翻译领域是一种全新的概念。
- ▶ 相比于其他模型，神经机器翻译模型在对语言学知识的依赖更少的前提下达到与之前方法相媲美的性能。从这之后，神经机器翻译方法正式走上了历史舞台，大量的研究者开始在这类方法上进一步研究和改进。到今天为止，神经机器翻译被广泛地应用在不同的工业场景下。

机器翻译现状与挑战

- ▶ 在特定条件下机器翻译的效果已经能够达到非常好的效果，甚至可以接近人工翻译效果。然而，在开放环境中，翻译效果还远没有达到直接使用的程度。根据机器翻译权威评测 WMT21 给出的人工评测结果，在新闻领域最好的中文到英文翻译系统评分也仅有 75 分左右 (满分 100 分)。机器翻译完全代替人工翻译还有很长的道路。
- ▶ 机器翻译虽然经过很多年的发展，目前在特定应用场景下已经能够有很好的效果，但是仍然面临挑战：
 - (1) 自然语言复杂度高。
 - (2) 翻译结果不可解释。
 - (3) 翻译结果评测困难。

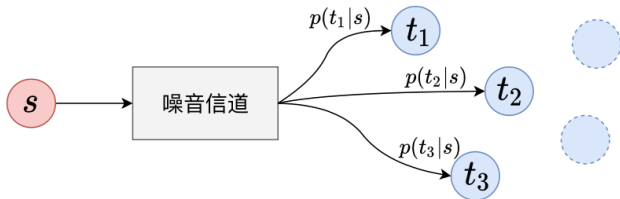
8.2 基于统计的机器翻译方法

基于统计的机器翻译方法

- ▶ IBM 机器翻译模型构建在噪声信道模型 (Noise Channel Model) 的基础之上。源语言句子 s 和候选的目标语言句子 t_i 通过一个噪声信道相连，在已知 s 和噪音信道性质的前提之下，就能够得到信源，也即目标语言的分布 $P(t|s)$ 。
- ▶ 机器翻译的过程本质上就是在给定源语言句子 s 的前提下，从分布 $P(t|s)$ 找到最有可能的目标语言 t 作为输出 $\hat{t} = \arg \max_t P(t|s)$ ，这一搜索过程也被称为解码。

源语言（信宿）

目标语言（信源）



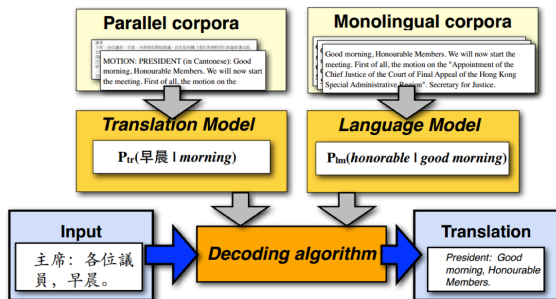
基于统计的机器翻译方法

- 在噪声信道基础框架下，统计机器翻译任务需要解决如下基本问题：

问题 1: 建模。 如何通过易于被计算机处理的数学模型对 $P(t|s)$ 进行合理地建模以刻画源语言和目标语言之间的关系。

问题 2: 训练。 如何从给定的平行语料库 (即源语言-目标语言对组成的语料集合) 中获得最优的模型参数。

问题 3: 解码。 如何从模型 $P(t|s)$ 中搜索出最优的目标语言序列 t 。



基于统计的机器翻译方法

- ▶ 首先，统计机器翻译模型的核心在于对 $P(t|s)$ 的定义，这一定义决定了模型性能的上限并且也是后续训练和解码的基础。IBM 模型通过贝叶斯公式对这一翻译概率做如下变换：

$$P(t|s) = \frac{P(s, t)}{P(s)} = \frac{P(s|t)P(t)}{P(s)}$$

通过上述变换，翻译模型 $P(t|s)$ 被分解为了三个部分：

- (1) 从目标语言指向源语言的翻译概率 $P(s|t)$ ；
- (2) 目标语言的语言模型 $P(t)$ ；
- (3) 源语言序列语言模型 $P(s)$ 。

基于统计的机器翻译方法

- ▶ 通过贝叶斯变换, $P(t|s)$ 和 $P(s|t)$ 只是翻译的方向不同, 建模难度并没有下降。其核心是为了引入目标语言的语言模型。这是由于 IBM 模型本质上是一种基于词的统计机器翻译模型, 仅通过翻译概率 $P(s|t)$ 很难有效地建模目标语言单词之间的相对位置关系, 即目标语言序列的流畅程度。相对应地, 引入语言模型 $P(t)$ 可以有效地缓解上述问题, 使语言序列更流畅。
- ▶ 此外, $P(s)$ 是一个不变量, 因此它不会影响到 $P(s|t)P(t)/P(s)$ 的相对大小, 也就不会影响到最终的解码过程, 在建模的过程当中 $P(s)$ 通常不需要 $P(s)$ 被计算, 可以省略:

$$\hat{t} = \arg \max_t P(t|s) = \arg \max_t \frac{P(s|t)P(t)}{P(s)} = \arg \max_t P(s|t)P(t)$$

基于统计的机器翻译方法

- ▶ 基于上述分析，IBM 模型的建模问题转换为如何建模翻译概率 $P(s|t)$ 以及语言模型 $P(t)$ 。
- ▶ 翻译概率 $P(s|t)$ 主要用于衡量源语言和目标语言之间的匹配程度。
- ▶ 然而，自然语言拥有极其庞大的潜在的组合方式。假设某种语言对应的词表大小为 10000，那么一个简单的长度为 10 的句子就对应着 $10000^{10} = 10^{40}$ 种不同的组合方式。基于任何已有的平行语料库，直接在句子层级对上述翻译概率进行估计，都会面临严重的数据稀缺问题。
- ▶ 因此，IBM 模型将句子层级的翻译概率进一步拆解为单词级别的对应关系的组合，从而缓解上述数据稀疏的问题，这一拆解过程又被称为词对齐。

基于统计的机器翻译方法

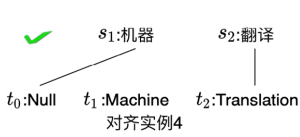
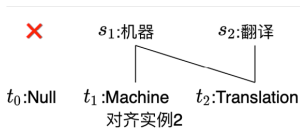
- ▶ 词对齐作为 IBM 模型构建的重要基础之一，描述了目标语言和源语言之间单词级别的对应关系。
- ▶ 给定源语言文本“机器翻译”，对应的目标语言翻译为“Machine Translation”。其中，“机器”一词对应“Machine”而“翻译”一词对应“Translation”。
使用记号 $a = \{a_1, \dots, a_m\}$ 表示这种对应关系，其中 a_j 表示源语言中的单词 s_j 和目标语言中的单词 t_{a_j} 存在对应关系。
举例来说，在对齐实例中， $a_1 = 1$, $a_2 = 2$ 。



基于统计的机器翻译方法

为了建模方便，IBM 模型对词对齐做了如下两个限制：

- ▶ 对于每一个源语言单词，至多只能对齐到一个目标语言单词上。对齐实例 2 中，源语言单词“机器”同时对应到了两个目标语言单词“Machine”和“Translation”，这就违反了上述 IBM 模型假设。而其余的对齐实例均满足这一假设。
- ▶ 存在一些源语言单词，它们可以对齐到一个额外增设的虚拟目标语言单词“Null”上，也即对空。对齐实例 4 中的“机器”一词就对应到了目标语言的“Null”上。对空情况的额外考虑并不是没有意义的。事实上，对空的现象在翻译的过程当中频繁出现，如虚词的翻译。



基于统计的机器翻译方法

- ▶ IBM 模型认为句子级别的翻译概率可以通过单词级别的翻译概率组合而成，并将词对齐作为一种隐变量整合到翻译概率的建模过程中。这样，原本较为困难的句子级别的建模问题就被分解为一个分步学习的问题：

$$P(s|t) = \sum_a P(s, a|t)$$

其中， $s = \{s_1, s_2, \dots, s_m\}$ 表示一个长度为 m 的源语言序列， $t = \{t_1, t_2, \dots, t_l\}$ 表示一个长度为 l 的目标语言序列， $a = \{a_1, a_2, \dots, a_m\}$ 表示源语言中每一个单词 s_j 对应的目标语言单词序号 a_j 。

基于统计的机器翻译方法

- 直接建模 $P(s|t)$ 仍然非常复杂，为了解决这个问题，IBM 模型对上述概率通过链式法则做了进一步展开，并为后续的简化做了准备。

$$P(s, a|t) = P(m|t) \prod_{j=1}^m P(a_j|a_1^{j-1}, s_1^{j-1}, m, t) P(s_j|a_1^j, s_1^{j-1}, m, t)$$

其中， a_1^{j-1} 表示源语言序列中前 $j-1$ 个单词的词对齐， s_1^{j-1} 表示源语言序列中的前 $j-1$ 个单词。

- 给定一个目标语言序列 t ，我们首先通过概率 $P(m|t)$ 估计源语言序列的长度 m 。接下来，通过 m 次循环从左向右依次生成源语言序列和它们的词对齐。在第 j 次循环当中，首先通过目标语言序列 t ，前 $j-1$ 次循环中生成的词对齐序列 a_1^{j-1} 以及源语言序列 s_1^{j-1} 产生当前位置的词对齐 a ，即 $P(a_j|a_1^{j-1}, s_1^{j-1}, m, t)$ 。接下来结合 a_j 进一步生成当前位置的源语言单词 s_j ，也即 $P(s_j|a_1^j, s_1^{j-1}, m, t)$ 。至此，翻译概率的建模实际上就被转换为源语言文本和词对齐的生成问题。

基于统计的机器翻译方法

仍然还存在两个迫切需要解决的问题:

- ▶ (1) 为了最终实现对翻译概率 $P(s|t)$ 的建模, 在公式 $P(s|t) = \sum_a P(s, a|t)$ 中需要对所有可能的词对齐进行求和。然而, 可能的词对齐的数量随着源语言序列的长度呈指数级别增长, 如何计算这一求和式是第一个需要被解决的问题。
- ▶ (2) 上页公式通过链式分解为建模 $P(s, a|t)$ 提供了一种可行的方向, 然而如何通过目标语言序列估计源语言序列的长度 $P(m|t)$, 以及如何建模源语言 $P(s_j|a_1^{j-1}, s_1^{j-1}, m, t)$ 和词对齐的生成过程 $P(a_j|a_1^{j-1}, s_1^{j-1}, m, t)$ 尚待解决。
- ▶ 对于上述两个问题的解决, 实际上对应着五个不同的 IBM 模型。

基于统计的机器翻译方法

- 为了衡量生成译文的流畅程度，IBM 模型引入了 n-gram 语言模型。它使用概率化的方法描述了句子的生成过程。
以 bi-gram 语言模型为例，一个目标语言序列的生成概率可以按照下式评估：

$$P_{lm}(t) = P_{lm}(t_1, \dots, t_l) = P(t_1) \times P(t_2|t_1) \times P(t_3|t_2) \times \dots \times P(t_l|t_{l-1})$$

其中 $P(t_i|t_{i-1})$ 均对应着语言模型的不同参数。这些参数的估计可以通过对语料库的极大似然估计完成。

$$P(t_i|t_{i-1}) = \frac{\text{单词对 } \langle t_{i-1}, t_i \rangle \text{ 出现的次数}}{\text{单词 } t_{i-1} \text{ 出现的次数}}$$

基于统计的机器翻译方法

- 在完成了对翻译概率 $P(s|t)$ 以及语言模型 $P(t)$ 的建模与优化之后，下一个需要解决的问题就是解码，即 $\arg \max_t P(s|t) \cdot P(t)$ 的问题。

代码 8.1: 贪婪解码算法

输入: 源语言句子序列 $s = \{s_1, s_2, \dots, s_m\}$

输出: 解码出的目标语言序列 $t = \{t_1, t_2, \dots, t_l\}$

初始化一个空集 $t_{best} = \emptyset$ 记录解码出的最优目标语言序列;

初始化一个布尔数组 u ，记录每个源语言单词是否已经被解码;

对于每个源语言单词 s_i ，获取其对应单词级别目标语言候选 π ;

for $i = 1$ **to** m **do**

 初始化一个空集 $h = \emptyset$ 记录当前最优解码序列的扩展;

for $j = 1$ **to** m **do**

if $u[j]$ 为假，也即当前单词没有被解码过 **then**

 将当前单词的候选翻译 $\pi[j]$ 添加到 h 当中;

end

end

 从当前的最优翻译扩展候选 h 中找到最优的翻译结果，并记为 t_{best} ;

 这一最优结果所对应的源语言单词被记录为已解码 $u[j] = True$;

end

return t_{best}

IBM 模型 I

- IBM 模型 I 假定源语言句子序列长度的生成概率服从均匀分布，即：

$$P(m|t) = \epsilon$$

其中， ϵ 是一个常量，在实际的建模和优化过程中，它通常被取特定的值来保证概率的归一化。

- 源语言中的每一个单词被认为是等可能地和目标语言中的所有单词对齐，即建模对齐概率：

$$P(a_j|a_1^{j-1}, s_1^{j-1}, m, t) = \frac{1}{l+1}$$

其中， $l+1$ 表示目标语言序列长度加上一个保留的虚拟单词“Null”。

- 当对齐关系明确之后，IBM 模型 I 假设当前时刻源语言单词 s_j 的生成只依赖于和它对齐的目标语言单词 t_{a_j} ：

$$P(s_j|\alpha_1^j, s_1^{j-1}, m, t) = f(s_j|t_{a_j})$$

$f(s_j|t_{a_j})$ 表示给定目标语言单词 t_{a_j} 之后，生成源语言单词 s_j 的概率。

IBM 模型 I

- 翻译概率 $P(s|t)$ 的计算

$$\begin{aligned}P(s|t) &= \sum_a P(s, a|t) \\&= P(m|t) \prod_{j=1}^m P(a_j|a_1^{j-1}, s_1^{j-1}, m, t) P(s_j|a_1^j, s_1^{j-1}, m, t) \\&= \sum_a \frac{\epsilon}{(1 + 1)^m} \prod_{j=1}^m f(s_j|t_{a_j})\end{aligned}$$

观察 IBM 模型 I 最终的建模结果可以发现，翻译概率 $P(s|t)$ 最终变成了在所有可能的词对齐的基础上，对单词对翻译概率的连乘。

- 一个长度为 m 的源语言序列的每一个单词有可能对齐到长度为 $l + 1$ 的目标语言的任何一个位置上。因此，对齐序列 $\sum_a (\cdot)$ 的求和一共要遍历 $(l + 1)^m$ 项。每一项都是一个 m 个单词级别翻译概率的乘积。公式的计算复杂度是 $O((l + 1)^m \cdot m)$ ，这在源语言序列长度 m 稍大的情况下几乎是不可能的。

IBM 模型 I

- 在实际的计算过程中，IBM 模型采用如下的计算技巧：

$$\begin{aligned}P(s|t) &= \sum_a \frac{\epsilon}{(l+1)^m} \prod_{j=1}^m f(s_j|t_{a_j}) \\&= \frac{\epsilon}{(l+1)^m} \sum_a \prod_{j=1}^m f(s_j|t_{a_j}) \\&= \frac{\epsilon}{(l+1)^m} \prod_{j=1}^m \sum_{i=1}^l f(s_j|t_i)\end{aligned}$$

通过将若干个连乘结果的加和转换为若干加和结果的连乘。计算复杂度降为 $O((l+1) \cdot m)$ 。计算复杂度的下降带来了更高的运行效率。

IBM 模型 I

► 优化目标:

$$L(f, \lambda) = \frac{\epsilon}{(l+1)^m} \prod_{j=1}^m \sum_{i=1}^l f(s_j | t_i) - \sum_{t_y} \lambda_{t_y} \left(\sum_{s_x} f(s_x | t_y) - 1 \right)$$

通过计算函数 $L(f, \lambda)$ 对参数 $f(s_x | t_y)$ 导数为 0 的位置得到其极值点。

$$f(s_u | t_v) = \lambda_{t_v}^{-1} \underbrace{\frac{\epsilon}{(l+1)^m} \prod_{j=1}^m \sum_{i=1}^l f(s_j | t_i)}_{P(\mathbf{s} | \mathbf{t})} \underbrace{\sum_{j=1}^m \delta(s_j, s_u) \cdot \sum_{i=1}^l \delta(t_i, t_v)}_{(s_u, t_v) \text{ 的配对次数}} \frac{f(s_u | t_v)}{\sum_{i=0}^l f(s_u | t_i)}$$

$\delta(x, y)$ 为指示函数, 当 $x = y$ 时, $\delta(x, y) = 1$, 否则 $\delta(x, y) = 0$

IBM 模型 I

- 将单词 t_v 翻译到 s_u 的期望频次表示为如下形式:

$$c_{\mathbb{E}}(s_u|t_v) \equiv \sum_{j=1}^m \delta(s_j, s_u) \cdot \sum_{i=1}^l \delta(t_i, t_v) \frac{f(s_u|t_v)}{\sum_{i=0}^l f(s_u|t_i)}$$

可得简化形式

$$f(s_u|t_v) = \frac{c_{\mathbb{E}}(s_u|t_v)}{\sum_{s'_u} c_{\mathbb{E}}(s'_u|t_v)}$$

通常，我们会有一个较大的平行语料数据集

$\mathcal{D} = \{(s_i, t_i)\}_{i=0}^n$, 单词对之间的期望频次可以通过下式计算:

$$c_{\mathbb{E}}(s'_u|t_v) = \sum_{i=0}^n c_{\mathbb{E}}(s_u|t_v; (s_i, t_i))$$

IBM 模型 II

- IBM 模型 II 认为词对齐存在着一定的倾向性。假设源语言单词 x_j 的对齐位置 a_j 的生成概率与它所在的位置 j 和源语言序列长度 m 以及目标语言序列长度 l 有关，形式化表示为：

$$P(a_j | a_1^{j-1}, s_1^{j-1}, m, t) \equiv a(a_j | j, m, l)$$

其中， $a(a_j | j, m, l)$ 表示源语言序列中第 j 个位置词对齐的生成概率，它被建模为 IBM 模型 II 中的一个需要学习的参数。

- 除了词对齐假设之外，其余的模型假设均与 IBM 模型 I 相同，将新的词对齐生成概率按照上一小节所述的建模过程能够得到 IBM 模型 II 的翻译建模表达式为：

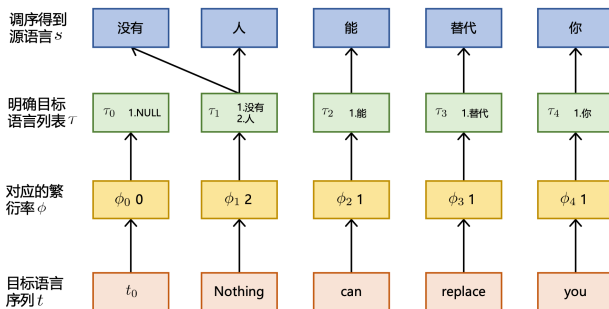
$$P(s|t) = \epsilon \prod_{j=1}^m \sum_{i=1}^l a(i|j, m, l) f(s_j | t_i)$$

IBM 模型 III

- ▶ IBM 模型 I 和 II 存在的一个共同的问题是将单词翻译的过程建模为了一个独立的过程，这就导致它们不能很好地描述多个源语言单词对齐到同一个目标语言单词的情况。
- ▶ IBM 模型 III 是一种基于繁衍率的模型，可在一定程度上解决上述问题。繁衍率 (Fertility) 是指每个目标语言单词生成源语言单词的个数。
- ▶ 模型首先确定每个目标语言单词的繁衍率 ϕ_i ，依据繁衍率确定目标语言对应的源语言单词是什么，这样就得到了每个目标语言单词所对应的源语言单词列表 τ_i 。最后将所有单词列表中的单词放置在合适的位置上就得到了源语言单词序列 s 。

IBM 模型 III

- 以单词 Nothing 的生成过程为例，它的繁衍率 $\phi_1 = 2$ ，这意味着在源语言当中它对应两个单词。确定 Nothing 的源语言单词列表为 $\tau_1 = \{\tau_{11} = \text{“没有”}, \tau_{12} = \text{“人”}\}$ 。最后，将单词列表中的单词放置在合适的位置，Nothing 对应的单词列表 τ_1 中的单词分别应该被放置在源语言序列中的第 1 和第 2 个位置。因此， $\pi_1 = \pi_{11} = 1, \pi_{12} = 2$ 。

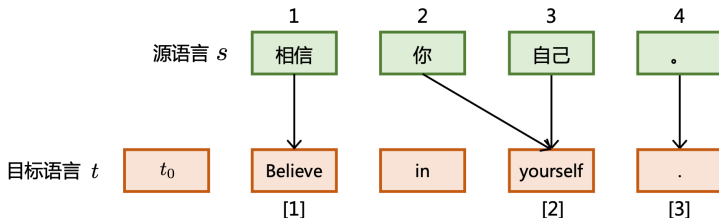


IBM 模型 IV

- ▶ 当一个目标语言单词对应多个源语言单词时，这些源语言单词往往会构成一个整体，即一个短语。然而前面所述的三个 IBM 模型并没有对与这种情况做特殊的设计，这就导致了源语言中的单词短语可能会被打散。
- ▶ IBM 模型 IV 将原本单词之间的对应关系拓宽到了概念 (Concept) 之间的对应。概念是指具有独立语法或语义的一组单词。
- ▶ IBM 模型 IV 将目标语言的概念约束为那些非空对齐的目标语言单词，且要求所有的目标语言概念都只能由一个单词构成。

IBM 模型 IV

- 例如，单词“yourself”是目标语言序列中的第 2 个概念，而单词“in”不是概念。在后面的叙述当中，标记目标语言中第 i 个概念所在的位置为 $[i]$ ， $\odot_i$ 表示目标语言中第 i 个概念对应的源语言位置的平均值，若这个平均值不是整数则向上取整。



IBM 模型 IV

- ▶ IBM 模型 IV 所做的修正主要体现在扭曲度的建模，对于 $[i]$ 对应的源语言单词列表中的第一个单词 $\tau_{[i]1}$ ，它的扭曲度计算公式如下：

$$P(\pi_{[i]1} = j | \pi_1^{[i]-1}, \tau_0^1, \phi_0^1, t) = d_1(j - \odot_{i-1} | A(t_{[i-1]}), B(s_j))$$

π_{ik} 表示目标语言序列中第 i 个单词所对应的源语言列表中的第 k 个单词的位置。对于列表中其他单词的扭曲度，计算：

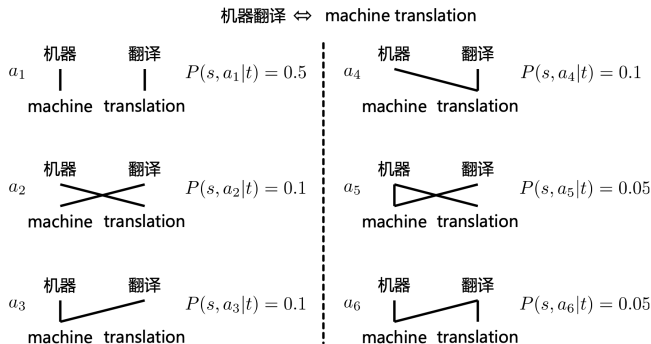
$$P(\pi_{[i]k} = j | \pi_{[i]1}^{k-1}, \pi_1^{[i]-1}, \tau_0^1, \phi_0^1, t) = d_{>1}(j - \pi_{[i]k-1} | B(s_j))$$

$A(\cdot)$ 和 $B(\cdot)$ 表示从源语言、目标语言单词向单词词类映射的函数。

- ▶ 关于扭曲度函数的改进思想是，在生成 $t_{[i]}$ 的第一个源语言单词时，要考虑平均位置 $\odot_{[i]}$ 和这个源语言单词之间的绝对距离。随后生成的单词所放置的位置则要考虑前一个放置完的单词的相对位置，以及当前源语言单词的词类。这个过程使得同一个目标语言单词所生成的源语言单词之间可以相互影响，从而避免了独立生成各个源语言单词所带来的冲突问题。

IBM 模型 V

- ▶ IBM 模型 V 针对词对齐的过程做了进一步的约束。它认为同一个源语言单词不应当由多个目标语言单词转换而来。
- ▶ 如图所示，前面 4 种词对齐方式都是合法的。然而，对于词对齐 a_5 和 a_6 来说，源语言单词“机器”和“翻译”分别对应着两个目标语言单词。



IBM 模型 V

- 为了约束这种情况的出现，IBM 模型 V 在放置每一个源语言单词时都会检查这个位置是否已经放置了其他单词。为了实现这一点，引入一个新的变量 $v(j, \tau_1^{[i]-1}, \tau_{[i]1}^{k-1})$ ，它表示在放置 $\tau_{[i]k}$ 之前，源语言的前 j 个位置还有多少空余。后续记这个变量为 v_j 。这样，对于单词 $[i]$ 所对应的源语言单词列表中的第一个单词 $\tau_{[i]1}$ 有：

$$P(\tau_{[i]1} = j | \pi_1^{[i]-1}, \tau_0^1, \phi_0^1, t) = d_1(v_j | B(s_j), v_{\odot_{i-1}}, v_m - (\phi_{[i]} - 1)) \cdot (1 - \delta(v_j, v_{j-1}))$$

对于其他单词 $\tau_{[i]k}, 1 < k \leq \phi_{[i]}$ ，有：

$$\begin{aligned} P(\pi_{[i]k} = j | \pi_{[i]1}^{k-1}, \pi_1^{[i]-1}, \tau_0^1, \phi_0^1, t) \\ = d_{>1}(v_j - v_{\pi_{[i]k-1}} | B(s_j), v_m - v_{\pi_{[i]k-1}} - \phi_{[i]} + k) \cdot (1 - \delta(v_j, v_{j-1})) \end{aligned}$$

$1 - \delta(v_j, v_{j-1})$ 是用来判断第 j 个位置是否为空。如果第 j 个位置为空，则 $v_j = v_{j-1}$ ，这样 $P(\tau_{[i]1} = j | \pi_1^{[i]-1}, \tau_0^1, \phi_0^1, t) = 0$ 。避免了词对齐的冲突问题。

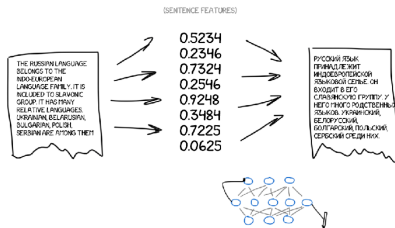
8.3 基于神经网络的机器翻译方法

基于神经网络的机器翻译方法

- ▶ 传统机器翻译方法高度依赖于繁杂的特征工程，合理特征的设计对系统构建者的语言学背景具有较高的要求，同时需要在不断地试错过程中修正。这些特征往往不能够完整地反映输入文本的语义。
 - ▶ 语言模型作为传统机器翻译模型的重要组成部分，为了降低模型复杂度而引入的马尔可夫假设使得上下文窗口之外的语义依赖无法被建模。
 - ▶ 从输入文本表示的角度来说，经典的词袋模型 (bag-of-words, BOW) 则忽略了词序对输入文本表示的影响。
- ▶ 传统机器翻译方法存在一些固有缺陷：
 - ▶ 以最具代表性的基于短语的统计机器翻译方法为例，翻译通过将输入的源语言切分成短语并替换为目标语言的过程完成，短语范围之外的长程依赖在这一过程中被完全忽略，进而造成翻译结果中的错误和不一致性。
 - ▶ 同时，为了提升翻译的准确性和流畅度，越来越多的功能模块不断被设计并添加到统计翻译模型当中（如语言模型、调序模型、长度调整模型等）。复杂的翻译组件使得系统的整体调优和稳定性受到一定程度的影响。

基于神经网络的机器翻译方法

- ▶ 神经网络模型作为一个强力的特征抽取器，能够自动地学习输入文本的最优表征，从而在很大程度上减少对领域知识的要求及繁琐的特征工程预处理步骤。
- ▶ 而以循环神经网络、Transformer 为代表的神经机器翻译方法能够有效地建模长程依赖，端到端的特性也使得系统的整体结构变得更加紧凑易于调整。
- ▶ 现代神经机器翻译模型大多依据序列到序列的方式对任务进行建模。给定源语言输入文本，训练目标是找到最合适的目标语言句子作为译文。



基于神经网络的机器翻译方法

- ▶ 形式化地来说，给定源语言序列 $S = \{s_1, s_2, \dots, s_n\}$ ，神经机器翻译模型试图找到具有最大条件概率 $P(T|S)$ 的目标语言序列 $T = \{t_1, t_2, \dots, t_m\}$ ， n 和 m 分别表示源语言和目标语言的长度。

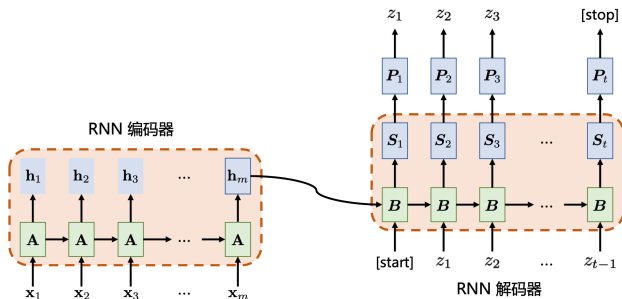
在生成目标语言句子的每个单词时，源语言和已经生成的目标语言信息会被使用。因此，神经机器翻译的整体过程可描述为：

$$\arg \max \prod_{i=1}^m P(t_i | t_{j < i}, S),$$

神经机器翻译利用不同的网络结构针对后验概率 $P(T|S)$ 进行建模，常见的结构包括卷积神经网络、循环神经网络、自注意力神经网络等。

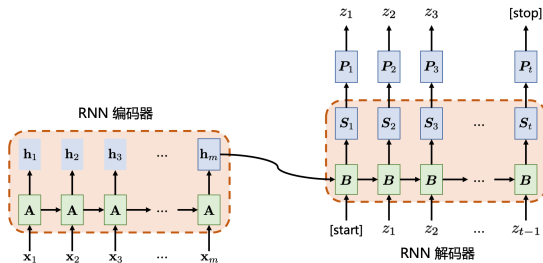
循环神经网络翻译模型

- ▶ 神经机器翻译模型大多基于序列到序列的架构，完成从源语言言到目标语言的转换过程。
- ▶ 不同神经机器翻译模型的主要区别在于编码器和解码器所采用的结构上的差异。
- ▶ 自然语言文本可以看做一种时间序列数据，因此一种常见做法是采用基于循环神经网络的结构完成对源语言文本的编码以及目标语言文本的生成。



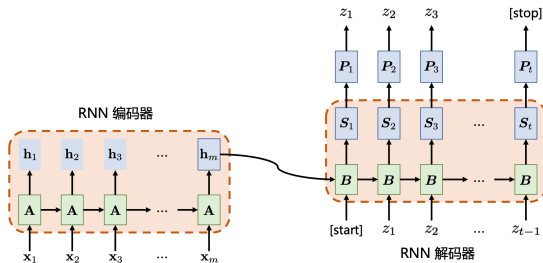
循环神经网络翻译模型

- 左侧为**编码器部分**，源语言单词按照其在文本序列中的先后顺序被依次送入到循环神经网络当中。在每个时间步 t 中，模型依据送入的源语言单词 x_t 对应修改维护其模型内部的隐状态 h_t ，这个隐状态编码了输入的源语言序列前 t 个时刻的所有必要信息。按照这种方式当 m 个输入全部被送入到编码器之后，所对应的 h_m 可以认为包含了源语言序列的所有信息。



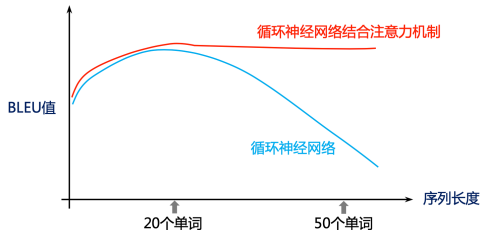
循环神经网络翻译模型

- 右半部分是 RNN 解码器部分，它接收编码器输出的编码源语言句子信息的向量 h_m 作为初始隐状态 s_0 。由于 RNN 的循环过程在每个时间步都要求一个输入单词，为了启动解码过程，一般会使用一个保留的特殊符号 “[Start]” 作为翻译开始的标记送入到 RNN 解码器当中，并解码出目标语言序列的第一个单词 z_1 。接下来， z_1 会作为下一个时刻的输入被送入到循环神经网络当中并按照不断迭代产生后续的预测。



循环神经网络翻译模型

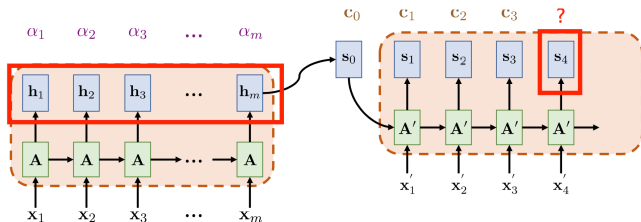
- 通过循环网络对源语言文本进行编码，并生成目标语言翻译结果的过程十分简单。然而，它仅仅使用一个定长的向量 h_m 编码整个源语言序列。这对于较短的源语言文本没有什么问题，但随着文本序列长度的逐渐加长，单一的一个向量 h_m 可能不足以承载源语言序列当中的所有信息。



蓝色的线代表上述简单循环神经网络性能随源语言文本长度的变化趋势。当文本长度在 20 个单词以内时，单一向量能够承载源语言文本中的必要信息。随着文本序列的进一步增加，翻译性能的评价指标 BLEU 的值就开始出现明显地下降。

循环神经网络翻译模型

- 引入注意力机制的循环机器翻译架构与基于简单循环网络的机器翻译模型大体结构相似，均采用循环神经网络作为编码器与解码器的实现。关键的不同点在于注意力机制的引入使得不再需要把原始文本中的所有必要信息压缩到一个向量当中。



循环神经网络翻译模型

- ▶ 具体来说，给定源语言序列经过编码器输出的向量序列 $h_1, h_2, \dots, h_m$ ，注意力机制旨在依据解码端翻译的需要，自适应地从这个向量序列中查找对应的信息。
- ▶ 与简单循环网络相类似，在 t 时刻，旨在通过 $t-1$ 时刻的隐状态 s_{t-1} 以及 t 时刻的输入 x'_t 维护循环隐状态并生成当前时刻目标语言翻译结果 x'_{t+1} 。
- ▶ 为了更高效地考虑源语言上下文语义来提高翻译质量，注意力机制通过计算一组匹配分数 $\{\text{score}_i\}_{i=1}^m$ 并利用 softmax 归一化为一组权重 $\{a_i\}_{i=1}^m$ 自适应地确定源语言中需要聚焦的部分。具体计算：

$$\{\alpha_i\}_{i=1}^m = \text{Softmax}(\{\text{score}_i\}_{i=1}^m)$$

$$\text{score}_i = \begin{cases} s_{t-1} h_i^T & \text{向量乘} \\ \cos(s_{t-1}, h_i^T) & \text{向量夹角} \\ s_{t-1} W h_i^T & \text{线性模型} \\ \tanh(W[s_{t-1}, h_i]) v^T & \text{单层网络} \end{cases}$$

循环神经网络翻译模型

- ▶ 基于上述权重能够得到生成译文 x'_t 所必要的源语言信息 c_t , 进一步, 可以将这部分源语言信息与当前时刻的输入 x'_t 拼接送入 RNN 作为新的输入:

$$c_i = \sum_i \alpha_i h_i$$

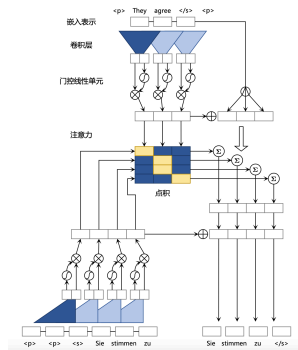
$$s_t = \tanh([x'_t, c_t]U + s_{t-1}W)$$

通过这样的修改, 在维护 RNN 任意时刻隐藏状态并生成译文的过程中, 能够自适应地考虑源语言中的哪部分信息需要被聚焦, 从而生成更加高质量的译文。

卷积神经网络翻译模型

卷积神经网络也是一种经典的神经网络结构，被广泛地使用在自然语言处理的各项任务当中。相较于循环神经网络来说，卷积神经网络每一步卷积操作并不依赖于前一时间步的计算结果，因而能够充分并行化以更好地利用 GPU 的计算资源。

- ▶ ConvS2S 的整体结构如图所示，它采用卷积神经网络作为编码器（图片上侧）和解码器（图片左下侧）的具体实现，完成对源语言和目标语言的特征提取。
- ▶ 这种模型结构使得每一层的网络计算可以完全并行化。
- ▶ 通过堆叠多层卷积结构，上下文窗口的范围得以不断扩大，从而逐渐建模输入文本中的远距离依赖。



卷积神经网络翻译模型

ConvS2S 主要由下述几个部件构成：

- ▶ **位置编码**：由于 ConvS2S 摒弃了循环结构，因此需要在输入层引入位置编码来标识输入序列中词与词之间的相对位置关系。
- ▶ **卷积层与门控线性单元**：这部分是编码器与解码器的实现模块，分别用于抽取源语言和目标语言的上下文语义特征。
- ▶ **残差连接**：编码器和解码器中堆叠的多层卷积结构当中增加了残差连接结构，直接连接每一层的输入与输出，从而提高信息传播效率，减小模型的优化难度。
- ▶ **多步注意力机制**：与循环神经机器翻译模型类似，也采用注意力机制自适应地从源语言端检索译文对应的源语言信息。不同的是，此处的注意力计算在解码器的每一层当中都会出现，因而被称为“多步”注意力。

卷积神经网络翻译模型

位置编码

- ▶ 由于 ConvS2S 不再使用基于循环的结构编码输入序列，因此模型失去了对于输入文本中词与词之间相对位置关系的感知。位置编码旨在重新给予模型这部分信息。
- ▶ 为了使得卷积模型能够感知到输入序列 (其在嵌入空间中的表示 $S = \{s_1, s_2, \dots, s_n\}$) 中单词的相对位置关系，使用额外的位置嵌入 $P = \{p_1, \dots, p_n\}$ 用于标识每个单词在句子中的绝对位置。
- ▶ 词嵌入和位置嵌入同时被使用作为编码器和解码器的输入 $E = \{s_1 + p_1, s_2 + p_2, \dots, s_n + p_n\}$ 。

卷积神经网络翻译模型

门控卷积结构

- ▶ 编码器和解码器均采用门控卷积结构作为建模源语言和目标语言的基本部件，这一部件由序列维度上的一维卷积运算和非线性门控机制结合而成。
- ▶ **卷积过程**能够有效地建模待处理文本中的局部上下文信息，而序列中的长程依赖问题则可以通过多层卷积结构的堆叠得到缓解。非线性门控机制使得我们能够建模输入视野下更加复杂的依赖关系。
- ▶ 具体来说，对于嵌入层输入的文本表示 $E \in \mathcal{R}^{n \times f}$ ，通过一个线性映射将维度转换到 d 维之后，我们能够得到卷积操作的每个上下文窗口的输入 $X \in \mathcal{R}^{k \times d}$ 。对其进行卷积运算：

$$A = X \times W_A + b_A \quad B = X \times W_B + b_B$$

对 X 进行卷积操作得到对应两组输出 A, B 后，基于门控线性单元的非线性变换被用作激活函数得到最终输出：

$$h = A \otimes \sigma(B)$$

残差连接 (residual connections) 被引入到模型构建当中。具体来说，每一层卷积单元的输入被直接连接到输出当中：

$$h^l = A^l \otimes \sigma(B^l) + h^{l-1}$$

卷积神经网络翻译模型

多步自注意力机制

- ▶ 解码器同样采用了堆叠的多层门控卷积结构完成对目标语言的解码，并在每一层门控卷积之后通过注意力机制参考源语言信息。
- ▶ 以解码器第 l 层第 i 个时间步的注意力计算为例，为了确定需要参考源语言中的哪部分信息，当前时刻的解码器状态 z_i^l 以及前一个时刻解码出的目标语言嵌入 g_i 用于作出决策的依据：

$$d_i^l = z_i^l W_d^l + b_d^l + g_i$$

$$z_i^l = \text{Conv}(s_i^l)$$

基于当前位置的状态依据 d_i^l ，目标语言位置 i 相对源语言第 j 个单词的注意力得分 a_{ij}^l 可以通过 d_i^l 和源语言编码器对应位置的输出 y_j 计算得到：

$$a_{ij}^l = \frac{\exp(d_i^l \cdot y_j)}{\sum_{t=1}^n \exp(d_i^l \cdot h_t)}$$

卷积神经网络翻译模型

多步自注意力机制

- 基于上述过程得到的注意力得分，可以对源语言不同位置的信息进行加权整合得到为了预测当前位置目标语言单词所需的依据：

$$c_i^l = \sum_{j=1}^m a_{ij}^l (h_j + e_j)$$

这里的源语言端同时利用了编码器的输出 h_j 以及对应位置的输入词嵌入 e_j 。这两者对应着更加全面的源语言信息，在实践中也证明十分有效。基于上述源语言信息，可以得到解码器端第 1 层的输出为：

$$s_i^{l+1} = c_i^l + z_i^l$$

卷积神经网络翻译模型

多步自注意力机制

► 上述多步注意力机制中的“多步”一词主要从两个方面体现。

1) 首先从多层卷积堆叠的角度来说，前一层中通过注意力机制动态地决定哪些相关信息需要被关注并传递到下一层当中，而下一层在计算对源语言不同位置的注意力得分过程中又会考虑到这些信息。

2) 从时间步的角度来说，在计算目标语言每个位置 i 的注意力分布时，前 k 个位置的注意力历史信息 $c_{i-k}^{l-1}, \dots, c_i^{l-1}$ 都会作为输入的一部分被考虑。使得在计算当前位置需要参考源语言中的哪些信息时能够有效地判断哪些信息在之前的时间步中已经被参考过了。

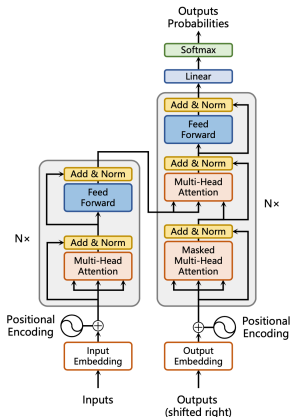
与之相对，在循环神经网络中，这一过程需要历经多个时间步中的非线性转换才能够被利用，这极有可能造成相关信息的丢失。

自注意力神经网络翻译模型

- ▶ **Transformer**是由谷歌在 2017 年提出的一种 Seq2Seq 架构。它的出现使得机器翻译的性能和效率迈向了一个新的阶段。它摒弃了循环结构，并完全通过注意力机制完成对源语言序列和目标语言序列全局依赖的建模。
- ▶ 在抽取每个单词的上下文特征时，Transformer 通过自注意力机制（self-attention）衡量上下文中每一个单词对当前单词的重要程度，在这个过程当中没有任何的循环单元参与计算。这种高度可并行化的编码过程使得模型的运行变得十分高效。

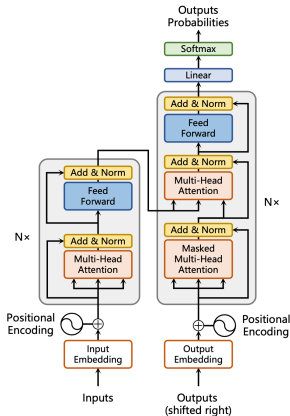
自注意力神经网络翻译模型

- ▶ 基于 Transformer 的机器翻译模型架构如图所示，左侧和右侧分别对应着 Seq2Seq 模型的编码器和解码器结构。
- ▶ 它们均由若干个基本的 Transformer 层组成（对应着图中的灰色框）。每个 Transformer 层都接收一个向量序列 $\{x_i\}_{i=1}^t$ 作为输入，并输出一个等长的向量序列作为输出 $\{y_i\}_{i=1}^t$ 。这里的 x_i 和 y_i 分别对应着文本序列中的一个单词的表示。而 y_i 是当前 Transformer 对输入 x_i 进一步整合其上下文语义后对应的输出。



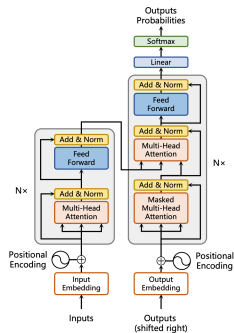
自注意力神经网络翻译模型

- ▶ **自注意力子层**：Multi-Head Attention。使用自注意力机制整合上下文语义，它使得序列中任意两个单词之间的依赖关系可以直接被建模而不基于传统的循环结构，从而更好地解决文本的长程依赖。
- ▶ **前馈子层**：的 Feed Forward。通过全连接层对输入文本序列中的每个单词表示进行更复杂的变换。
- ▶ **残差连接**：Add。它是一条分别作用在上述两个子层当中的直连通路，被用于连接它们的输入与输出。从而使得信息流动更加高效，有利于模型的优化。
- ▶ **层标准化**：Norm。作用于上述两个子层的输出表示序列中，对表示序列进行层标准化操作，同样起到稳定优化的作用。



自注意力神经网络翻译模型

- ▶ 解码器的每个 Transformer 层的第一个自注意力子层额外增加了**注意力掩码 (Masked Multi-Head Attention)**。额外增加的掩码是用来掩盖后续文本信息，以防模型在训练阶段直接看到后续文本序列进而无法得到有效地训练。
- ▶ 解码器端还额外增加了一个**多头注意力模块**，需要注意的是它同时接收来自编码器端的输出以及当前 Transformer 层第一个掩码注意力层的输出。它的作用是在翻译的过程当中，为了生成合理的目标语言序列需要观测待翻译的源语言序列是什么。
- ▶ 待翻译的源语言文本，首先经过编码器端的每个 Transformer 层对其上下文语义的层层抽象，最终输出每一个源语言单词上下文相关的表示。**解码器端以自回归的方式生成目标语言文本**，即每个时间步 t 参考编码器端输出的所有源语言文本表示以及前 $t - 1$ 个时刻生成的目标语言文本生成当前时刻的目标语言单词。



自注意力神经网络翻译模型

- ▶ **位置编码**：对于待翻译的文本序列，首先通过输入嵌入层将每个单词转换为其相对应的向量表示。在送入编码器端建模其上下文语义之前，是在词嵌入中加入位置编码这一特征。由于 Transformer 不再使用基于循环的方式建模文本输入，序列中不再有任何信息能够提示模型单词之间的相对位置关系。因此补充这部分信息是十分必要的。
- ▶ 具体来说，序列中每一个单词所在的位置都对应一个实值向量。这一实值向量会与单词表示对应相加并送入到后续模块中做进一步处理。在训练的过程当中，模型会自动地学习到如何利用这部分位置信息。为了得到不同位置对应的编码，Transformer 使用不同频率的正余弦函数如下所示：

$$\text{PE}(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/d}}\right), \quad \text{PE}(\text{pos}, 2i + 1) = \cos\left(\frac{\text{pos}}{10000^{2i/d}}\right)$$

其中，pos 表示单词所在的位置， $2i$ 和 $2i + 1$ 表示位置编码向量中的对应维度， d 则对应位置编码的总维度。

- ▶ **通过上面这种方式计算位置编码有这样几个好处**：首先，正余弦函数的范围是在 $[-1, +1]$ ，导出的位置编码与原词嵌入相加不会使得结果偏离过远而破坏原有单词的语义信息。其次，依据三角函数的基本性质，可以得到第 $\text{pos} + k$ 个位置的编码是第 pos 个位置的编码的线性组合，这就意味着位置编码中蕴含着单词之间的距离信息。

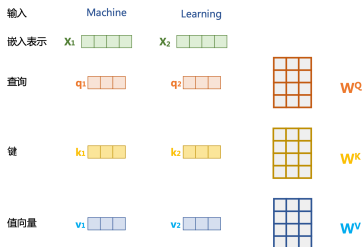
自注意力神经网络翻译模型

- ▶ **自注意力子层**：自注意力（Self-Attention）机制是基于Transformer 的机器翻译模型的基本操作，在源语言的编码和目标语言的生成中频繁地被使用以建模源语言、目标语言任意两个单词之间的依赖关系。
- ▶ 给定由单词语义嵌入及其位置编码叠加得到的输入表示 $\{\mathbf{x}_i \in \mathbb{R}^d\}_{i=1}^t$ ，为了实现对上下文语义依赖的建模，进一步引入在自注意力机制中涉及到的三个元素：
查询 q_i (Query)，键 k_i (Key)，值 v_i (Value)。
- ▶ 在编码输入序列中每一个单词的表示的过程中，这三个元素用于计算上下文单词所对应的权重得分。直观地说，这些权重反映了在编码当前单词的表示时对于上下文不同部分所需要的关注程度。

自注意力神经网络翻译模型

- ▶ 通过三个线性变换 $W^Q \in \mathbb{R}^{d \times d_k}$, $W^K \in \mathbb{R}^{d \times d_k}$, $W^V \in \mathbb{R}^{d \times d_v}$ 将输入序列中的每一个单词表示 x_i 转换为其对应的 $q_i \in \mathbb{R}^{d_k}$, $k_i \in \mathbb{R}^{d_k}$, $v_i \in \mathbb{R}^{d_v}$ 向量。为了得到编码单词 x_i 时所需要关注的上下文信息，通过位置 i 查询向量与其他位置的键向量做点积得到匹配分数： $q_i \cdot k_1, q_i \cdot k_2, \dots, q_i \cdot k_t$ 。
- ▶ 经过 Softmax 归一化为概率之后，与其他位置的值向量相乘来聚合我们希望关注的上下文信息，并最小化不相关信息的干扰：

$$Z = \text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$



自注意力神经网络翻译模型

- 为了进一步增强自注意力机制聚合上下文信息的能力，提出了多头 (Multi-head) 自注意力的机制，以从关注上下文的不同侧面。上下文中每一个单词的表示 x_i 经过多组线性映射 $\{W_j^Q, W_j^K, W_j^V\}_{j=1}^N$ 到不同的表示子空间中。
- Attention 计算公式会在不同的子空间中分别计算并得到不同的上下文相关的单词序列表示 $\{Z_j\}_{j=1}^N$ 。最终，线性变换 $W^O \in \mathbb{R}^{(Nd_v) \times d}$ 用于综合不同子空间中的上下文表示并形成自注意力层最终的输出 $\{x_i \in \mathbb{R}^d\}_{i=1}^t$ 。

自注意力神经网络翻译模型

- **前馈子层**：前馈子层接受自注意力子层的输出作为输入，并通过一个带有 Relu 激活函数的两层全连接网络对输入进行更加复杂的非线性变换。实验证明，这一非线性变换会对模型最终的性能产生十分重要的影响。

$$\text{FFN}(x) = \text{Relu}(xW_1 + b_1)W_2 + b_2$$

$$\text{Relu}(x) = \max(0, x)$$

其中 W_1, b_1, W_2, b_2 表示前馈子层的参数。

实验结果表明，增大前馈子层隐状态的维度有利于提升最终翻译结果的质量，因此，前馈子层隐状态的维度一般比自注意力子层要大。

自注意力神经网络翻译模型

- **残差连接与层标准化**：编码器和解码器均由 6 层 Transformer 层组成，每一层当中都包含复杂的非线性映射，导致模型的训练困难。因此，研究者在 Transformer 层中进一步引入了残差连接与层标准化技术以进一步提升训练的稳定性。具体地，残差连接主要是指使用一条直连通道直接将对子层的输入连接到输出上去，从而避免由于网络过深在优化过程中潜在的梯度消失问题：

$$x^{l+1} = f(x^l) + x^l$$

其中 x^l 表示第 l 层的输入， $f(\cdot)$ 表示一个映射函数。此外，为了进一步使得每一层的输入输出范围稳定在一个合理的范围内，层标准化技术被进一步引入 Transformer 的每一层当中：

$$\text{LN}(x) = g \cdot \frac{x - \mu}{\sigma} + b$$

其中 μ 和 σ 分别表示均值和方差，用于将数据平移缩放到均值为 0，方差为 1 的标准分布， g 和 b 是可学习的参数。**层标准化技术可以有效地缓解优化过程中潜在的不稳定、收敛速度慢等问题。**

8.4 机器翻译评估

机器翻译评估

Bilingual Evaluation Understudy (BLEU)¹

- ▶ 在机器翻译领域，我们需要一个指标来衡量机器翻译的结果与专业人工翻译结果的差异，一般情况下，如果需要比较好的结果都需要专业的翻译人员介入评估模型翻译的好坏，但是这样需要大量的人力参与。
- ▶ 评机器翻译结果通常使用 BLEU(双语互译质量辅助工具)。对于模型预测序列中任意的子序列，BLEU 考察这个子序列是否出现在标签序列中。
- ▶ 计算这个指标，需要使用机器翻译好的文本 (称作 candidate docs) 以及一些专业翻译人员翻译的文本 (称作 reference docs)。本质上讲 BLEU 就是用来衡量机器翻译文本与参考文本之间的相似程度的指标，取值范围在 0-1，取值越靠近 1 表示机器翻译结果越好。

¹BLEU: a Method for Automatic Evaluation of Machine Translation, ACL, 2002.

机器翻译评估

- ▶ 我们取测试语料库的修改后的精度 (modified precision) 分数的几何平均值, 然后将结果乘以指数短语惩罚因子 (brevity penalty factor)。在计算精度之前, 只进行大小写折叠 (case folding) 的文本规范化。首先使用长度为 N 的 n -gram 和正权重 w_n 的几何平均值计算修改后的 n -gram 精度 p_n (modified n -gram precisions) 的几何平均值。
- ▶ 设候选翻译的长度为 c , 有效参考语料库的长度为 r 。可计算短语惩罚 BP, 以及 BLEU 指标

$$BP = \begin{cases} 1 & \text{if } c > r \\ e^{(1-r/c)} & \text{if } c \leq r \end{cases}$$

$$BLEU = BP \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right)$$

在对数域中,

$$\log BLEU = \min(1 - r/c, 0) + \sum_{n=1}^N w_n \log p_n$$

在论文基准模型中使用 $N = 4$ 和均匀权重 $w_n = 1/N$ 。

8.5 机器翻译语料库

机器翻译语料库

- ▶ WMT 数据集: 一个以英语为主的多语言机器翻译数据集, 涉及英中、英德翻译等多种任务, 数据来源于新闻、医学、翻译等领域。
- ▶ IWSLT 数据集: 一个来自于 TED 演讲的文本翻译数据集, 语料规模较小, 涉及英德、英中翻译等任务。
- ▶ NIST 数据集: 一个新闻翻译领域的高质量数据集, 评测集包括 4 句参考译文, 涉及中英、英捷翻译等任务。
- ▶ TVsub: 一个抽取自电视剧字幕的翻译数据集, 适合于对话中的长距离上下文依赖研究, 涉及中英翻译任务。
- ▶ Flickr30k: 多模态机器翻译的主流数据集之一, 包含 31783 张图片, 每张图片对应 5 个语句标注, 涉及英德翻译任务。
- ▶ Multi30k: 多模态机器翻译的主流数据集之一, 包含 31014 张图片, 每张图片对应 5 个语句标注, 设计英德、英法翻译任务。
- ▶ IAPRTC-12: 多模态翻译的主流数据集之一, 包含 20000 张图片及其对应标注, 涉及英德翻译任务。
- ▶ IKEA: 多模态翻译领域的主流数据集之一, 包含 3600 张图片及其对应标注, 涉及英德、英法翻译任务。

机器翻译语料库

大规模双语平行语料

- ▶ NewsCommentaryCorpus: 该语料库爬取自 ProjectSyndicate 网站的政治经济评论，涉及中文、英语等 12 个语种，64 个语言对的双语数据。
- ▶ CWMTCorpus: 由中国计算机翻译研讨会社区收集和共享的中英平行预料数据，来源于新闻、电影、小说、政府文档等多个领域。
- ▶ Common Crawl Corpus: 爬取自互联网网页的数据，涵盖捷克语、德语、法语、俄语等 4 种语言到英语的平行语料。
- ▶ Europarl Corpus: 该语料库的数据来源是欧洲议会记录，涵盖了保加利亚语、捷克语等 20 种欧洲语言到英语的平行语料。
- ▶ ParaCrawl Corpus: 该语料库的数据来源依然是互联网爬取，包含了 23 种欧洲语言到英语的双语语料。
- ▶ United Nations Parallel Corpus: 该语料库的数据来源是联合国公共领域的官方记录和其他会议文件，涵盖了阿拉伯语、英语、西班牙语、法语、俄语、汉语等 6 种联合国正式语言。
- ▶ TED Corpus: 该语料库的数据来源是 TED 大会演讲在其网站公开的自 2007 年以来的演讲字幕，以及超过 100 种语言的翻译版本。

机器翻译语料库

大规模双语平行语料

- ▶ OpenSubtitle: 收集自 opensubtitles 电影字幕网站，是一个涵盖了 62 种语言、1782 个语言对的平行语料的大规模数据集。
- ▶ WikititlesCorpus: 来源维基百科的标题，涵盖了古吉拉特语等 14 个语种，11 个语言对的平行预料数据。
- ▶ CzEng: 来源是欧洲法律、信息技术和小说领域，涵盖了捷克语和英语的平行语料。
- ▶ YandexCorpus: 爬取自互联网网页，涵盖了俄语和英语的平行语料。
- ▶ Tilde MODELCorpus: 多个来自于经济、新闻、政府、旅游等门户网站的数据集组成，涵盖了欧洲多种语言的开放数据。
- ▶ Setimes Corpus: 东南欧时报的新闻报道，涵盖了克罗地亚语、阿尔巴尼亚语等 9 种巴尔干语言，72 个语言对的双语数据。
- ▶ TVsub: 电视剧字幕的中英文对话的语料，主要用于对话和长距离上下文信息依赖的研究。
- ▶ Recipe Corpus: 一个包含 10 万多个句对的由 Cookpad 公司创建的日英食谱语料库。

本章小结

第八章 机器翻译

8.1 机器翻译概述

8.2 基于统计的机器翻译方法

8.3 基于神经网络的机器翻译方法

8.4 机器翻译评估

8.5 机器翻译语料库